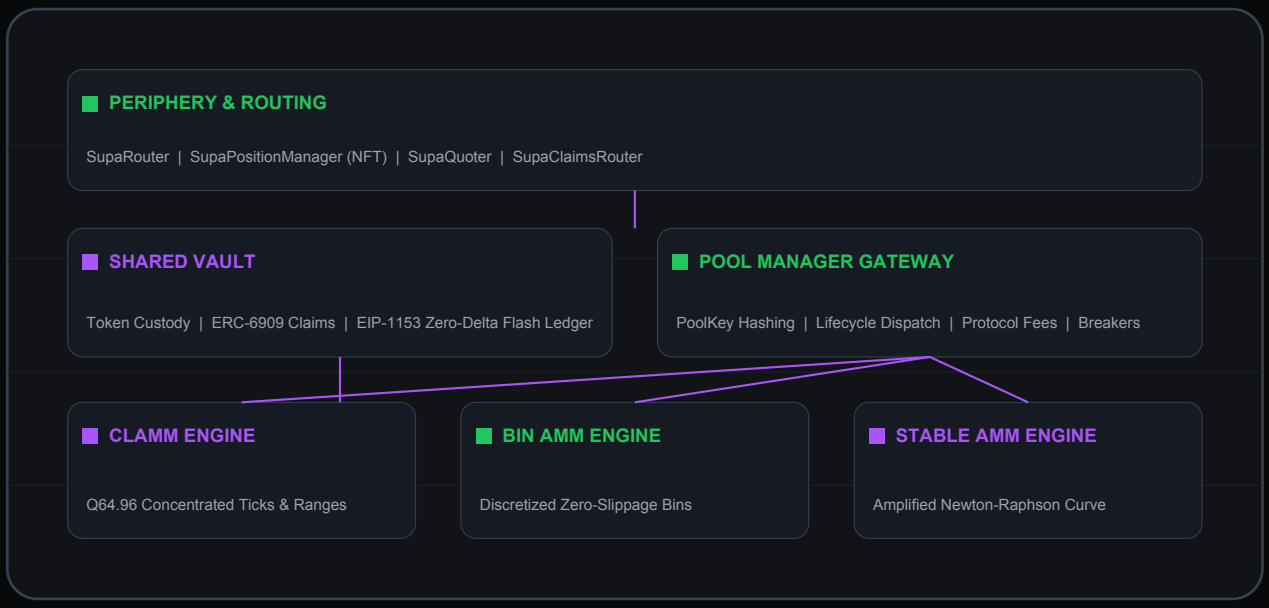
SUPADAO DEX / OFFICIAL PROTOCOL PUBLICATION

Protocol & Architecture

Shared settlement, specialized curve engines, explicit trust boundaries



VERSION 1.0.0 · 15 OCTOBER 2026 · UNDER TECHNICAL REVIEW

Institutional Notice: Repository-derived documentation based on audited smart contract implementations. No independent external audit or public deployment was confirmed.

Document Control & Verification Status

This technical publication is reproducible directly from verified Markdown specifications in the SupaDAO DEX repository. It describes inspected smart contract code and does not constitute legal, tax, or investment advice.

Contents & Architecture Sections

01 Purpose
02 Topology
03 Pool identity
04 Execution and accounting
05 Curve-specific design
06 Fees and plugins
07 Controls
08 Integration and residual risk

Version: 1.0.0 | Date: 2026-10-15 | Status: Under Technical Review

Purpose

This guide isolates the technical architecture from the broader white paper. SupaDAO DEX is coordinated by a shared pool manager and vault while curve-specific engines retain their own state and mathematics.

Topology

`SupaPoolManager` is the common entry point for pool initialization, liquidity changes, swaps, donations and protocol fees. `SupaVault` custodies assets, maintains ERC-6909 claims and enforces transient settlement. The manager selects `CLAMMEngine`, `BinAMMEngine` or `StableAMMEngine` based on `PoolKey.curveType`.

Pool identity

A pool ID hashes ordered currencies, fee tier, tick/bin spacing, plugin address and curve type: `PoolId = keccak256(abi.encode(PoolKey))` Two pools sharing currencies are still independent if any field differs. The singleton architecture does not automatically combine their depth.

Execution and accounting

Periphery contracts open a vault unlock callback. Each engine returns a packed signed token delta. The manager converts that result into transient obligations. Input transfer, output withdrawal and claims operations must clear the obligations before the callback ends (EIP-1153 net zero delta).

Curve-specific design

- CLAMM: Q64.96 prices, initialized ticks, bitmap search, range positions and inside fee growth.
- Bin AMM: Active-bin inventory, discrete bin price and per-bin fee growth. Cross-bin traversal is not implemented in current source.
- Stable AMM: Two reserves, default amplification and iterative invariant solving. No decimal or rate normalization was found.

Fees and plugins

Pool fee units are 1e-6 (hundredths of a bip). LP fees accrue through engine fee growth; protocol fees accrue by currency. Optional plugins receive lifecycle callbacks based on a permission bitmask. The dynamic-fee plugin is experimental and has a known stored-tick update limitation.

Controls

Circuit breakers pause vault, pool or curve scopes. Guardians can pause; owner controls unpause. A timelock supports delayed operations and roles. The vault is UUPS upgradeable; the manager is not UUPS in the reviewed source.

Integration and residual risk

Integrators must source canonical deployment addresses, reconstruct exact `PoolKey` values, quote current state and impose slippage/deadline checks. Primary risks are custom math, hooks, token behavior, administration, oracle limitations, depegs, liquidity gaps and MEV. No independent audit or public deployment was verified.